

Users Service — Documentation technique

DANEBCYS

Microservice de gestion des profils, adresses, favoris et administration des utilisateurs pour **DANEBCYS**. Les favoris sont stockés dans MongoDB. Les adresses dans PostgreSQL. L'authentification est déléguée au Auth Service via appels HTTP internes.

Table des matières

1. Vue d'ensemble
 2. Architecture du projet
 3. Communication avec le Auth Service
 4. Gestion des profils
 5. Gestion des adresses (PostgreSQL)
 6. Gestion des favoris (MongoDB)
 7. Gestion Premium / Abonnements
 8. Administration des utilisateurs
 9. Rate Limiter basé sur le token
 10. Schéma de base de données
 11. Endpoints API
 12. Installation et lancement
 13. Variables d'environnement
 14. Exemples d'utilisation (cURL)
-

1. Vue d'ensemble

Le **Users Service** est le microservice de gestion utilisateurs de DANEBCYS :

Responsabilité	Stockage
Profils (lecture/écriture)	Via Auth Service (API)
Adresses	PostgreSQL (propre DB)
Favoris	MongoDB Atlas
Premium / Abonnements	Via Auth Service (API)
Admin utilisateurs	Via Auth Service (API)

Principes architecturaux :

- Chaque microservice est **propriétaire exclusif** de ses données
- Aucun accès direct aux tables d'un autre service
- Toute interaction inter-services passe par API
- L'authentification est validée via POST /internal/validate-token du Auth Service

Stack :

- Express.js (JavaScript pur)
 - PostgreSQL avec driver natif pg (adresses)
 - MongoDB avec driver natif mongodb (favoris)
 - Aucune librairie d'auth — validation par appel HTTP au Auth Service
 - Rate limiter maison (même pattern que Auth Service)
-

2. Architecture du projet

```
Users-service/
├─ src/
│   ├─ config/
│   │   ├─ database.js          # Pool PostgreSQL natif (adresses)
│   │   ├─ mongodb.js          # Connexion MongoDB Atlas (favoris)
│   │   └─ env.js              # Variables d'environnement
│   ├─ controllers/
│   │   ├─ profile.controller.js # Profil utilisateur
│   │   ├─ address.controller.js # CRUD adresses
│   │   ├─ favorite.controller.js # CRUD favoris (MongoDB)
│   │   └─ admin.controller.js  # Administration (rôle admin)
│   ├─ middlewares/
│   │   ├─ auth.js             # Validation token via Auth Service
│   │   ├─ rateLimiter.js      # Rate limiter maison (par user ID)
│   │   └─ admin.js            # Vérification rôle admin
│   ├─ routes/
│   │   ├─ user.routes.js       # Routes utilisateur /api/v1/users
│   │   ├─ admin.routes.js      # Routes admin /api/v1/users/admin
│   │   └─ internal.routes.js   # Routes internes /internal
│   ├─ services/
│   │   ├─ authClient.js        # Client HTTP natif vers Auth Service
│   │   ├─ profile.service.js   # Logique profil (proxy Auth Service)
│   │   ├─ address.service.js   # Logique adresses (PostgreSQL)
│   │   └─ favorite.service.js  # Logique favoris (MongoDB)
│   ├─ utils/
│   │   └─ errors.js            # Classes d'erreur (AppError, 400-429)
│   └─ app.js                  # Config Express (helmet, cors, routes)
├─ public/
│   ├─ index.html              # Interface de test (navigateur)
│   └─ test.js                 # Logique client (appels API)
```

```
└─ init.sql # Schéma PostgreSQL (table addresses)
└─ .env
└─ package.json
└─ server.js # Point d'entrée
```

3. Communication avec le Auth Service

Fichier : src/services/authClient.js

Principe Le Users Service ne possède **aucune donnée utilisateur** directement. Toutes les opérations sur les profils passent par les routes internes du Auth Service, authentifiées via X-Service-Key.

Client HTTP natif On utilise le module http natif de Node.js (pas d'axios, pas de node-fetch) pour rester cohérent avec l'architecture du projet.

```
callAuth(method, path, body)
→ HTTP request vers AUTH_SERVICE_URL
→ Header X-Service-Key: INTER_SERVICE_KEY
→ Timeout: 5 secondes
→ Parse JSON response
```

Endpoints Auth Service utilisés

Méthode	Route interne Auth Service	Usage
POST	/internal/validate-token	Validation du JWT
GET	/internal/users/:id	Lire un profil
PUT	/internal/users/:id	Modifier un profil
GET	/internal/users	Lister les utilisateurs
PUT	/internal/users/:id/role	Changer le rôle
PUT	/internal/users/:id/premium	Changer le niveau premium
DELETE	/internal/users/:id	Soft delete
PUT	/internal/users/:id/restore	Restaurer un user
GET	/internal/roles	Lister les rôles disponibles

Middleware d'authentification

1. Extraire le Bearer token du header Authorization
2. Appeler POST /internal/validate-token avec le token
3. Si valid: true → req.user = { id, email, username, role }
4. Si valid: false → 401 Unauthorized

Chaque requête au Users Service génère un appel HTTP au Auth Service pour valider le token. En production, un cache court (ex: 30s) pourrait être ajouté.

4. Gestion des profils

Fichiers : src/services/profile.service.js + src/controllers/profile.controller.js

Lecture du profil

```
GET /api/v1/users/me
→ authenticate middleware
→ authClient.getUser(req.user.id)
→ Retourne le profil complet formaté en camelCase
```

Modification du profil

```
PUT /api/v1/users/me
Body: { username?, firstName?, lastName?, phone?, country? }
→ Seuls les champs fournis sont mis à jour
→ Appel à authClient.updateUser()
→ Les contraintes d'unicité (email, username) sont vérifiées par Auth Service
```

Suppression du compte

```
DELETE /api/v1/users/me
→ Soft delete via authClient.softDeleteUser()
→ L'utilisateur est marqué deleted = TRUE dans Auth Service
→ Peut être restauré par un admin
```

Format de réponse

```
{
  "user": {
    "id": "uuid",
    "username": "jean_dupont",
    "email": "jean@example.com",
    "phone": "+33612345678",
    "firstName": "Jean",
    "lastName": "Dupont",
    "emailVerified": true,
    "phoneVerified": false,
    "role": "user",
```

```

    "premiumLevel": "none",
    "country": "France",
    "createdAt": "2026-02-24T12:00:00.000Z",
    "updatedAt": "2026-02-24T12:00:00.000Z",
    "lastLoginAt": "2026-02-24T12:00:00.000Z"
  }
}

```

5. Gestion des adresses (PostgreSQL)

Fichiers : src/services/address.service.js + src/controllers/address.controller.js

Base de données Les adresses sont stockées dans la **propre base PostgreSQL** du Users Service (danebcys_users). Le user_id est un UUID qui référence l'utilisateur dans le Auth Service, mais **sans contrainte FK cross-service** (CDC 3.2).

Opérations

Route	Description
GET /me/addresses	Lister mes adresses (triées par défaut)
POST /me/addresses	Ajouter une adresse
GET /me/addresses/:id	Détail d'une adresse
PUT /me/addresses/:id	Modifier une adresse
DELETE /me/addresses/:id	Supprimer (soft delete)
PUT /me/addresses/:id/default	Définir comme adresse par défaut

Adresse par défaut Un seul is_default = TRUE par utilisateur est garanti par un **index unique partiel** :

```

CREATE UNIQUE INDEX idx_addresses_default_per_user
  ON addresses(user_id) WHERE is_default = TRUE AND deleted = FALSE;

```

Quand on définit une nouvelle adresse par défaut :

1. UPDATE addresses SET is_default = FALSE WHERE user_id = ... AND deleted = FALSE
2. UPDATE addresses SET is_default = TRUE WHERE id = ... AND user_id = ...

Format de réponse

```

{
  "address": {
    "id": "uuid",
    "label": "Maison",

```

```
    "street": "12 rue de la Paix",
    "city": "Paris",
    "zipCode": "75001",
    "country": "France",
    "isDefault": true,
    "createdAt": "2026-02-24T12:00:00.000Z",
    "updatedAt": "2026-02-24T12:00:00.000Z"
  }
}
```

6. Gestion des favoris (MongoDB)

Fichiers : src/services/favorite.service.js + src/controllers/favorite.controller.js

Pourquoi MongoDB ? Le CDC (5.2) spécifie que les favoris doivent être dans MongoDB : données flexibles, volumétriques, avec un schéma simple et des opérations fréquentes (ajout/suppression).

Collection favorites

```
{
  "_id": "ObjectId",
  "user_id": "uuid-string",
  "ad_id": "id-annonce",
  "added_at": "Date",
  "deleted": false
}
```

Index

Index	Type	Description
{ user_id: 1, ad_id: 1 }	Unique	Partiel (deleted: false) — pas de doublons
{ user_id: 1 }	Standard	Recherche par utilisateur

Opérations

Route	Description
GET /me/favorites	Lister mes favoris (paginé)
GET /me/favorites/count	Nombre total de favoris
GET /me/favorites/:adId/check	Vérifier si une annonce est en favori
POST /me/favorites/:adId	Ajouter aux favoris
DELETE /me/favorites/:adId	Retirer des favoris (soft delete)

Soft delete et restauration Quand un favori est retiré, il est marqué `deleted: true`. Si l'utilisateur re-ajoute le même favori, le document existant est restauré (`deleted: false`, `added_at` mis à jour) au lieu d'en créer un nouveau.

Connexion MongoDB

```
mongodb+srv://...@cluster0.qvm6etd.mongodb.net/  
Database: danebcys  
Collection: favorites
```

La connexion utilise le driver `mongodb` natif (pas `Mongoose`), cohérent avec l'approche du projet.

7. Gestion Premium / Abonnements

Fichier : `src/services/profile.service.js`

Niveaux disponibles (ENUM PostgreSQL)

Niveau	Description	Justificatif requis
<code>none</code>	Utilisateur standard	Non
<code>premium</code>	Abonnement premium	Non
<code>premium_avancee</code>	Abonnement premium avancé	Non
<code>etudiant</code>	Tarif étudiant	Oui (obligatoire)

Flux de demande

1. User appelle `PUT /api/v1/users/me/premium` Body: `{ premiumLevel: "etudiant", studentProof: "https://..." }`
2. Validation côté Users Service :
 - Niveau dans la liste autorisée
 - Si "etudiant", justificatif obligatoire
3. Appel à Auth Service `PUT /internal/users/:id/premium`
4. Auth Service met à jour les colonnes `premium_level` et `student_proof`

En production, une étape de validation admin serait ajoutée (CDC 6.5).

8. Administration des utilisateurs

Fichiers : `src/controllers/admin.controller.js` + `src/routes/admin.routes.js`

Accès Toutes les routes admin sont protégées par :

1. `authenticate` — validation du token via Auth Service

2. userLimiter — rate limiting par user ID
3. requireAdmin — vérifie que req.user.role === 'admin'

Opérations admin

Route	Description
GET /admin/roles	Lister les rôles disponibles
GET /admin/users	Lister les utilisateurs (paginé)
GET /admin/users/:id	Détail d'un utilisateur
PUT /admin/users/:id/role	Changer le rôle
PUT /admin/users/:id/premium	Changer le niveau premium
DELETE /admin/users/:id	Soft delete
PUT /admin/users/:id/restore	Restaurer un utilisateur supprimé

Filtres de liste (GET /admin/users)

Query param	Description
page	Numéro de page (défaut: 1)
limit	Éléments par page (défaut: 20, max: 100)
search	Recherche sur email, username, nom, prénom
role	Filtrer par rôle (user, vendeur, assistance, admin)
deleted	true = supprimés, all = tous, défaut = actifs

Rôles (CDC 7)

ID	Nom	Description
1	user	Utilisateur standard
2	vendeur	Vendeur (CRUD produits, stats)
3	assistance	Support client, litiges, tickets
4	admin	Super administrateur, accès complet

9. Rate Limiter basé sur le token

Fichier : src/middlewares/rateLimiter.js

Même implémentation maison que le Auth Service :

- Stockage en mémoire (Map)
- Clé = user:\${req.user.id} (après authentification)
- Nettoyage automatique toutes les 60 secondes
- Headers : X-RateLimit-Limit, X-RateLimit-Remaining, X-RateLimit-Reset

Limiter	Config	Cible
userLimiter	100 req / 15 min	Toutes les routes

10. Schéma de base de données

PostgreSQL — Table addresses

Base : danebcys_users (propre au Users Service)

Colonne	Type	Description
id	UUID (PK)	gen_random_uuid()
user_id	UUID	Référence user Auth Service (pas de FK)
label	VARCHAR(100)	Libellé (Maison, Bureau, etc.)
street	VARCHAR(255)	Rue (NOT NULL)
city	VARCHAR(100)	Ville (NOT NULL)
zip_code	VARCHAR(20)	Code postal (NOT NULL)
country	VARCHAR(100)	Pays (NOT NULL)
is_default	BOOLEAN	Adresse par défaut (unique partiel)
deleted	BOOLEAN	Soft delete
created_at	TIMESTAMPTZ	Date de création
updated_at	TIMESTAMPTZ	Trigger auto-update

Index :

- idx_addresses_default_per_user : UNIQUE sur user_id WHERE is_default = TRUE AND deleted = FALSE
- idx_addresses_user_id : sur user_id
- idx_addresses_deleted : sur deleted

Trigger : trg_addresses_updated_at → BEFORE UPDATE → updated_at = NOW()

MongoDB — Collection favorites

Base : danebcys (MongoDB Atlas)

Champ	Type	Description
_id	ObjectId	Auto-généré par MongoDB
user_id	String	UUID de l'utilisateur
ad_id	String	ID de l'annonce
added_at	Date	Date d'ajout
deleted	Boolean	Soft delete (défaut: false)

Index :

- { user_id: 1, ad_id: 1 } unique (partialFilterExpression: { deleted: false })
- { user_id: 1 }

11. Endpoints API

Routes utilisateur — /api/v1/users Toutes protégées par JWT (via Auth Service) + Rate Limiter.

Méthode	Route	Description
GET	/me	Mon profil
PUT	/me	Modifier mon profil
DELETE	/me	Supprimer mon compte
PUT	/me/premium	Demander un niveau premium
GET	/me/adresses	Lister mes adresses
POST	/me/adresses	Ajouter une adresse
GET	/me/adresses/:id	Détail d'une adresse
PUT	/me/adresses/:id	Modifier une adresse
DELETE	/me/adresses/:id	Supprimer une adresse
PUT	/me/adresses/:id/default	Définir comme adresse par défaut
GET	/me/favorites	Lister mes favoris (paginé)
GET	/me/favorites/count	Nombre de favoris
GET	/me/favorites/:adId/check	Vérifier si en favori
POST	/me/favorites/:adId	Ajouter aux favoris
DELETE	/me/favorites/:adId	Retirer des favoris

Routes admin — `/api/v1/users/admin` Protégées par JWT + Rate Limiter + rôle admin.

Méthode	Route	Description
GET	/roles	Lister les rôles
GET	/users	Lister les utilisateurs
GET	/users/:id	Détail d'un utilisateur
PUT	/users/:id/role	Changer le rôle
PUT	/users/:id/premium	Changer le niveau premium
DELETE	/users/:id	Soft delete
PUT	/users/:id/restore	Restaurer un utilisateur

Routes internes — `/internal` Protégées par X-Service-Key (timing-safe).

Méthode	Route	Description
GET	/favorites/:userId/adId	Vérifier un favori (inter-service)
GET	/favorites/:userId/count	Compter les favoris (inter-service)

Routes utilitaires

Méthode	Route	Protection	Description
GET	/health	Aucune	Health check

12. Installation et lancement

Prérequis

- Node.js >= 18
- PostgreSQL >= 14
- MongoDB Atlas (ou MongoDB local)
- **Auth Service en fonctionnement** (port 3001)

Installation

```
cd Users-service
npm install

# Créer la base de données PostgreSQL
psql -U postgres -c "CREATE DATABASE danebcys_users;"
```

Lancement

```
# Développement (auto-reload)
npm run dev

# Production
npm start
```

Le schéma PostgreSQL (init.sql) et les index MongoDB sont créés **automatiquement** au démarrage.

Ordre de démarrage global

1. **pepper-primary** (port 3098)
2. **pepper-service** (port 3099)
3. **Auth Service** (port 3001) — requis par Users Service
4. **Users Service** (port 3002)

Séquence de démarrage

1. Connexion au pool PostgreSQL (SELECT 1)
2. Initialisation du schéma DB (init.sql → table addresses)
3. Connexion MongoDB Atlas + création des index (favorites)
4. Démarrage du serveur Express (app.listen)

13. Variables d'environnement

Users Service (.env)

Variable	Description	Défaut
PORT	Port du serveur	3002
NODE_ENV	Environnement	development
PG_HOST	Hôte PostgreSQL	localhost
PG_PORT	Port PostgreSQL	5432
PG_DATABASE	Nom de la base (adresses)	danebcys_users
PG_USER	Utilisateur PostgreSQL	postgres

PG_PASSWORD	Mot de passe PostgreSQL	postgres
MONGO_URI	URI MongoDB Atlas	<i>requis</i>
MONGO_DB_NAME	Nom de la base MongoDB	danebcys
AUTH_SERVICE_URL	URL du Auth Service	http://localhost:3001
INTER_SERVICE_KEY	Clé partagée avec Auth Service	<i>requis</i>
RATE_LIMIT_WINDOW_MS	Fenêtre du rate limit (ms)	900000
RATE_LIMIT_MAX_REQUESTS	Max requêtes par fenêtre	100

14. Exemples d'utilisation (cURL)

Tous les exemples nécessitent un Access Token obtenu depuis le Auth Service.

Health check

```
curl http://localhost:3002/health
```

Mon profil

```
curl http://localhost:3002/api/v1/users/me \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Modifier mon profil

```
curl -X PUT http://localhost:3002/api/v1/users/me \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{
    "firstName": "Jean",
    "lastName": "Dupont",
    "phone": "+33612345678",
    "country": "France"
  }'
```

Demander un niveau premium

```
curl -X PUT http://localhost:3002/api/v1/users/me/premium \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{ "premiumLevel": "etudiant", "studentProof":
"http://example.com/justificatif.pdf" }'
```

Supprimer mon compte

```
curl -X DELETE http://localhost:3002/api/v1/users/me \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Ajouter une adresse

```
curl -X POST http://localhost:3002/api/v1/users/me/addresses \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer ACCESS_TOKEN" \
  -d '{
    "label": "Maison",
    "street": "12 rue de la Paix",
    "city": "Paris",
    "zipCode": "75001",
    "country": "France",
    "isDefault": true
  }'
```

Lister mes adresses

```
curl http://localhost:3002/api/v1/users/me/addresses \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Définir une adresse par défaut

```
curl -X PUT
http://localhost:3002/api/v1/users/me/addresses/ADDRESS_ID/default \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Ajouter un favori

```
curl -X POST http://localhost:3002/api/v1/users/me/favorites/AD_ID \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Lister mes favoris

```
curl "http://localhost:3002/api/v1/users/me/favorites?page=1&limit=20" \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Vérifier si une annonce est en favori

```
curl http://localhost:3002/api/v1/users/me/favorites/AD_ID/check \
  -H "Authorization: Bearer ACCESS_TOKEN"
```

Retirer un favori

```
curl -X DELETE http://localhost:3002/api/v1/users/me/favorites/AD_ID \
-H "Authorization: Bearer ACCESS_TOKEN"
```

[Admin] Lister les utilisateurs

```
curl
"http://localhost:3002/api/v1/users/admin/users?page=1&limit=20&search=jean"
-H "Authorization: Bearer ADMIN_ACCESS_TOKEN"
```

[Admin] Changer le rôle d'un utilisateur

```
curl -X PUT http://localhost:3002/api/v1/users/admin/users/USER_ID/role \
-H "Content-Type: application/json" \
-H "Authorization: Bearer ADMIN_ACCESS_TOKEN" \
-d '{ "roleId": 2 }'
```

[Admin] Supprimer un utilisateur

```
curl -X DELETE http://localhost:3002/api/v1/users/admin/users/USER_ID \
-H "Authorization: Bearer ADMIN_ACCESS_TOKEN"
```

[Admin] Restaurer un utilisateur

```
curl -X PUT http://localhost:3002/api/v1/users/admin/users/USER_ID/restore
-H "Authorization: Bearer ADMIN_ACCESS_TOKEN"
```

Interface de test Ouvrir <http://localhost:3002> dans le navigateur. Coller un Access Token (obtenu depuis le Auth Service) dans la barre en haut, puis tester tous les endpoints graphiquement.

Récapitulatif architecture

Couche	Technologie
Authentification	Déléguée au Auth Service (appel HTTP interne)
Profils	Proxy vers Auth Service /internal/users
Adresses	PostgreSQL propre (danebcys_users)
Favoris	MongoDB Atlas (collection favorites)
Rate limiting	Maison, basé sur user ID, en mémoire (Map)
Inter-services	X-Service-Key + SHA-256 timingSafeEqual

Injection SQL	Requêtes paramétrées (\$1, \$2, ...) exclusivement
Injection NoSQL	Driver natif MongoDB (pas de string concat)
Headers sécurité	helmet (CSP, X-Frame-Options, etc.)
Soft delete	addresses (PostgreSQL) + favorites (MongoDB)

Librairies auth utilisées : aucune.